

Chương 5: Cơ sở dữ liệu và SQL

5.1. Cơ bản về SQL

- SQL là gì ?
 - SQL (Structured Query Language) là ngôn ngữ truy vấn có cấu trúc . Nó là công cụ được thiết kế nhằm để quản lý dữ liệu sử dụng trong nhiều lĩnh vực , cho phép bạn truy cập và thực hiện các thao tác lấy các hàng hoặc sửa đổi các hàng, trích xuất ,tạo, xóa dữ liệu....
- Các chức năng của SQL
 - Cho phép chúng ta truy cập Database theo nhiều cách khác nhau , nhờ sử dụng các lệnh.
 - Người dùng có thể truy cập dữ liệu từ cơ sở dữ liệu quan hệ.
 - SQL còn cho phép người sử dụng miêu tả dữ liệu.
 - Cho phép người dùng định nghĩa dữ liệu thao tác nó khi cần thiết trong một Database.
 - Bạn có thể tạo , xóa Database và bảng...

Tóm tắt một số các câu lệnh định nghĩa dữ liệu SQL



- CREATE TABLE: tạo một bảng mới trong csdl
- INSERT: Chèn dữ liệu mới thêm vào CSDL
- UPDATE: Sửa đổi, cập nhật dữ liệu trong cơ sở dữ liệu.
- DELETE: Xóa dữ liệu từ cơ sở dữ liệu.
- SELECT: Trích xuất bản ghi cụ thể từ một hoặc nhiều bảng

Tạo các cấu trúc bảng bằng SQL

5.1.1. CREATE TABLE và INSERT

- Ví dụ tạo một bảng `users` chứa `user_id`, `name`, `num_friends`:

```
users = [[0, "Hero", 0],  
         [1, "Dunn", 2],  
         [2, "Sue", 3],  
         [3, "Chi", 3]]
```

- Trong SQL, câu lệnh CREATE TABLE có cú pháp như sau:

```
CREATE TABLE users (  
    user_id INT NOT NULL,  
    name VARCHAR(200),  
    num_friends INT);
```

Tạo các cấu trúc bảng bằng SQL

- Ta có thể thêm dữ liệu bằng câu lệnh insert:

```
INSERT INTO users (user_id, name, num_friends) VALUES (0, 'Hero', 0);
```

- Câu lệnh SQL trên sẽ thêm mới 1 hàng trong bảng users với các trường dữ liệu tương ứng : user_id = 0, name = 'Hero', num_friends = 0

Tạo các cấu trúc bảng với NotQuiteABase

- Để thao tác với NotQuiteABase ta cần các thư viện Tuple, Sequence, List, Any, Callable, Dict, Iterator trong python

```
from typing import Tuple, Sequence, List, Any, Callable, Dict, Iterator
from collections import defaultdict
```

- Một số key word chúng ta sẽ sử dụng nhiều :

```
Row = Dict[str, Any]                # Một hàng dữ liệu trong database
WhereClause = Callable[[Row], bool] # Thuộc tính trong 1 hàng
HavingClause = Callable[[List[Row]], bool] # Thuộc tính trong nhiều hàng
```

Tạo các cấu trúc bảng với NotQuiteABase

- Để tạo một bảng NotQuiteABase ta cần tên bảng, danh sách cột và các kiểu dữ liệu của cột tương ứng là tham số cho phương thức khởi tạo.

```
class Table:
    def __init__(self, columns: List[str], types: List[type]) -> None:
        assert len(columns) == len(types), "# of columns must == # of types"

        self.columns = columns          # Names of columns
        self.types = types              # Data types of columns
        self.rows: List[Row] = []      # (no data yet)
```

- Chúng ta cần thêm 1 phương thức để lấy kiểu dữ liệu của cột trong bảng

```
def col2type(self, col: str) -> type:
    idx = self.columns.index(col)      # Find the index of the column,
    return self.types[idx]             # and return its type.
```

Tạo các cấu trúc bảng với NotQuiteABase

- Chúng ta cần một phương thức để insert data vào bảng:

```
def insert(self, values: list) -> None:
    # Check for right # of values
    if len(values) != len(self.types):
        raise ValueError(f"You need to provide {len(self.types)} values")

    # Check for right types of values
    for value, typ3 in zip(values, self.types):
        if not isinstance(value, typ3) and value is not None:
            raise TypeError(f"Expected type {typ3} but got {value}")

    # Add the corresponding dict as a "row"
    self.rows.append(dict(zip(self.columns, values)))
```

- Trong phương thức này ta cần kiểm tra số lượng cột và kiểu dữ liệu của từng cột có khớp với bảng mà ta insert data hay không

Tạo các cấu trúc bảng với NotQuiteABase

- Để dễ dàng thao tác với table hơn chúng ta sẽ coi table như một danh sách các hàng data. Vì vậy chúng ta cần các phương thức truy cập vào các hàng.

```
def __getitem__(self, idx: int) -> Row:  
    return self.rows[idx]
```

```
def __iter__(self) -> Iterator[Row]:  
    return iter(self.rows)
```

```
def __len__(self) -> int:  
    return len(self.rows)
```

- Phương thức `__getitem__` : Trả về data của 1 bản ghi
- Phương thức `__iter__` : Sử dụng khi duyệt đến bản ghi tiếp theo
- Phương thức `__len__` : Trả về số lượng bản ghi trong table

Tạo các cấu trúc bảng với NotQuiteABase

- Cuối cùng chúng ta cần 1 phương thức để mô tả dữ liệu các bản ghi trong table:

```
def __repr__(self):  
    """Pretty representation of the table: columns then rows"""  
    rows = "\n".join(str(row) for row in self.rows)  
  
    return f"{self.columns}\n{rows}"
```

- Phương thức này sẽ duyệt qua tất cả các bản ghi trong table và trả về dữ liệu của các bản ghi dưới dạng string
- Format của data sẽ là hàng đầu tiên mô tả tên các cột , các hàng kế tiếp sẽ là dữ liệu các bản ghi trong table

Tạo các cấu trúc bảng với NotQuiteABase

- Bây giờ hãy thử tạo mới một table và tạo ra các bản ghi cho table.

```
# Constructor requires column names and types
users = Table(['user_id', 'name', 'num_friends'], [int, str, int])
users.insert([0, "Hero", 0])
users.insert([1, "Dunn", 2])
users.insert([2, "Sue", 3])
users.insert([3, "Chi", 3])
users.insert([4, "Thor", 3])
users.insert([5, "Clive", 2])
users.insert([6, "Hicks", 3])
users.insert([7, "Devin", 2])
users.insert([8, "Kate", 2])
users.insert([9, "Klein", 3])
users.insert([10, "Jen", 1])
```

Tạo các cấu trúc bảng với NotQuiteABase

- Nếu như chúng ta sử dụng câu lệnh `print(users)` đầu ra của chúng ta chính là phương thức mô tả dữ liệu đã cài đặt ở trên

```
['user_id', 'name', 'num_friends']  
{ 'user_id': 0, 'name': 'Hero', 'num_friends': 0}  
{ 'user_id': 1, 'name': 'Dunn', 'num_friends': 2}  
{ 'user_id': 2, 'name': 'Sue', 'num_friends': 3}  
...
```

Cập nhật dữ liệu cho các bản ghi trong bảng

5.1.2. UPDATE

- Để thay đổi dữ liệu trong bảng, ta dùng lệnh UPDATE.
- Ví dụ cú pháp của lệnh UPDATE như sau:

```
UPDATE users  
SET num_friends = 3  
WHERE user_id = 1;
```

- Lưu ý rằng điều kiện WHERE là không bắt buộc trong câu lệnh UPDATE. Nếu không có điều kiện WHERE, thì câu lệnh UPDATE sẽ được thực hiện trên tất cả các bản ghi của bản đó.

Cập nhật dữ liệu với NotQuiteABase

- Ta sẽ tạo một hàm UPDATE vào NotQuiteABase.

```
def update(self,
            updates: Dict[str, Any],
            predicate: WhereClause = lambda row: True):
    # First make sure the updates have valid names and types
    for column, new_value in updates.items():
        if column not in self.columns:
            raise ValueError(f"invalid column: {column}")

        typ3 = self.col2type(column)
        if not isinstance(new_value, typ3) and new_value is not None:
            raise TypeError(f"expected type {typ3}, but got {new_value}")

    # Now update
    for row in self.rows:
        if predicate(row):
            for column, new_value in updates.items():
                row[column] = new_value
```

Cập nhật dữ liệu với NotQuiteABase

- Sau đó ta có thể dễ dàng làm điều này.

```
assert users[1]['num_friends'] == 2           # Original value

users.update({'num_friends' : 3},             # Set num_friends = 3
             lambda row: row['user_id'] == 1) # in rows where user_id == 1

assert users[1]['num_friends'] == 3           # Updated value
```

Xóa các bản ghi khỏi bản



5.1.3. DELETE

- Ví dụ cú pháp của lệnh DELETE như sau:

```
DELETE FROM users WHERE user_id = 1;
```

- Giống với UPDATE, câu lệnh WHERE không bắt buộc, nếu điều kiện đó không được chỉ rõ thì tất cả các hàng của bảng sẽ được xóa

Xóa dữ liệu với NotQuiteABase

- Ta sẽ tạo một hàm DELETE vào NotQuiteABase.

```
def delete(self, predicate: WhereClause = lambda row: True) -> None:
    """Delete all rows matching predicate"""
    self.rows = [row for row in self.rows if not predicate(row)]
```

- Nếu ta điền giá trị cho predicate thì sẽ chỉ xóa các hàng thỏa mãn đấy. Nếu không cung cấp, predicate mặc định trả về True và sẽ xóa mọi hàng.
- Ví dụ:

```
# We're not actually going to run these
users.delete(lambda row: row["user_id"] == 1) # Deletes rows with user_id == 1
users.delete()                                # Deletes every row
```

Câu lệnh truy vấn trong SQL

5.1.4. SELECT

- Ví dụ cú pháp của lệnh SELECT như sau:

```
SELECT * FROM users;           -- get the entire contents
SELECT * FROM users LIMIT 2;   -- get the first two rows
SELECT user_id FROM users;     -- only get specific columns
SELECT user_id FROM users WHERE name = 'Dunn'; -- only get specific rows
```

- Ta cũng có thể tính toán trên câu lệnh SELECT

```
SELECT LENGTH(name) AS name_length FROM users;
```

truy vấn với NotQuiteABase

- Ta sẽ tạo hàm SELECT.

```
def select(self,
            keep_columns: List[str] = None,
            additional_columns: Dict[str, Callable] = None) -> 'Table':

    if keep_columns is None:          # If no columns specified,
        keep_columns = self.columns  # return all columns

    if additional_columns is None:
        additional_columns = {}

    # New column names and types
    new_columns = keep_columns + list(additional_columns.keys())
    keep_types = [self.col2type(col) for col in keep_columns]

    # This is how to get the return type from a type annotation.
    # It will crash if `calculation` doesn't have a return type.
    add_types = [calculation.__annotations__['return']]
                  for calculation in additional_columns.values()]

    # Create a new table for results
    new_table = Table(new_columns, keep_types + add_types)

    for row in self.rows:
        new_row = [row[column] for column in keep_columns]
        for column_name, calculation in additional_columns.items():
            new_row.append(calculation(row))
        new_table.insert(new_row)

    return new_table
```

truy vấn với NotQuiteABase

- Ta cũng sẽ cần WHERE và LIMIT.

```
def where(self, predicate: WhereClause = lambda row: True) -> 'Table':  
    """Return only the rows that satisfy the supplied predicate"""  
    where_table = Table(self.columns, self.types)  
    for row in self.rows:  
        if predicate(row):  
            values = [row[column] for column in self.columns]  
            where_table.insert(values)  
    return where_table
```

```
def limit(self, num_rows: int) -> 'Table':  
    """Return only the first `num_rows` rows"""  
    limit_table = Table(self.columns, self.types)  
    for i, row in enumerate(self.rows):  
        if i >= num_rows:  
            break  
        values = [row[column] for column in self.columns]  
        limit_table.insert(values)  
    return limit_table
```

truy vấn với NotQuiteABase

- Giờ ta sẽ thao tác dễ dàng tương tự SQL

```
# SELECT * FROM users;
all_users = users.select()
assert len(all_users) == 11

# SELECT * FROM users LIMIT 2;
two_users = users.limit(2)
assert len(two_users) == 2

# SELECT user_id FROM users;
just_ids = users.select(keep_columns=["user_id"])
assert just_ids.columns == ['user_id']

# SELECT user_id FROM users WHERE name = 'Dunn';
dunn_ids = (
    users
    .where(lambda row: row["name"] == "Dunn")
    .select(keep_columns=["user_id"])
)
assert len(dunn_ids) == 1
assert dunn_ids[0] == {"user_id": 1}

# SELECT LENGTH(name) AS name_length FROM users;
def name_length(row) -> int: return len(row["name"])

name_lengths = users.select(keep_columns=[],
                             additional_columns = {"name_length": name_length})
assert name_lengths[0]['name_length'] == len("Hero")
```

GROUP BY



5.1.5. GROUP BY

- GROUP BY gộp nhóm các hàng đã được chọn ra dựa trên một hoặc nhiều thuộc tính và tạo ra các giá trị tổng hợp như MIN, MAX, COUNT và SUM
- Ví dụ: ta muốn tìm số lượng người dùng và user_id nhỏ nhất cho mỗi độ dài tên.

```
SELECT LENGTH(name) as name_length,  
       MIN(user_id) AS min_user_id,  
       COUNT(*) AS num_users  
FROM users  
GROUP BY LENGTH(name);
```

GROUP BY



- Đối với GROUP BY thì SQL cũng hỗ trợ mệnh đề HAVING – nó có chức năng tương tự như mệnh đề WHERE
- Ví dụ: ta muốn biết số lượng bạn bè trung bình của những người dùng có tên bắt đầu bằng chữ cái cụ thể với số lượng bạn bè trung bình lớn hơn 1

```
SELECT SUBSTR(name, 1, 1) AS first_letter,  
       AVG(num_friends) AS avg_num_friends  
FROM users  
GROUP BY SUBSTR(name, 1, 1)  
HAVING AVG(num_friends) > 1;
```

GROUP BY

- Để thêm GROUP BY vào NotQuiteABase Tables ta sẽ thêm phương thức `group_by` với các tham số :
 - `Group_by_columns` : Danh sách các cột mà bạn muốn nhóm lại
 - `Aggregates` : Một từ điển của các hàm tổng hợp mà bạn muốn chạy trên mỗi nhóm
 - `Having` : Một vị từ tùy chọn được gọi là có hoạt động trên nhiều hàng

```
def group_by(self,  
    group_by_columns: List[str],  
    aggregates: Dict[str, Callable],  
    having: HavingClause = lambda group: True) -> 'Table':
```

GROUP BY

- Tiếp theo chúng ta cần nhóm các bản ghi trong table lại theo cột
- Để làm được điều này ta cần dùng kiểu dữ liệu dict trong python làm danh sách các bản ghi đã được nhóm (grouped_rows)
- Sau đó ta sẽ duyệt các hàng của table và thêm vào grouped_rows theo các cột cần nhóm trong group_by_columns ở tham số truyền vào

```
grouped_rows = defaultdict(list)

# Populate groups
for row in self.rows:
    key = tuple(row[column] for column in group_by_columns)
    grouped_rows[key].append(row)
```

GROUP BY

- Việc tiếp theo ta cần làm là tạo một Table mới với các cột mới mà ta muốn nhóm cũng kiểu dữ liệu tương ứng của chúng

```
# Result table consists of group_by columns and aggregates
new_columns = group_by_columns + list(aggregates.keys())
group_by_types = [self.col2type(col) for col in group_by_columns]
aggregate_types = [agg.__annotations__['return']
                    for agg in aggregates.values()]
result_table = Table(new_columns, group_by_types + aggregate_types)
```

GROUP BY

- Cuối cùng ta chỉ cần duyệt qua danh sách hàng đã nhóm (grouped_rows) rồi chèn các bản ghi tương ứng đó vào bảng kết quả đã tạo ở trên với điều kiện having ở đầu vào và trả về bảng kết quả

```
for key, rows in grouped_rows.items():
    if having(rows):
        new_row = list(key)
        for aggregate_name, aggregate_fn in aggregates.items():
            new_row.append(aggregate_fn(rows))
        result_table.insert(new_row)

return result_table
```

GROUP BY

- Đây là toàn bộ code phương thức group_by

```
def group_by(self,
              group_by_columns: List[str],
              aggregates: Dict[str, Callable],
              having: HavingClause = lambda group: True) -> 'Table':

    grouped_rows = defaultdict(list)

    # Populate groups
    for row in self.rows:
        key = tuple(row[column] for column in group_by_columns)
        grouped_rows[key].append(row)

    # Result table consists of group_by columns and aggregates
    new_columns = group_by_columns + list(aggregates.keys())
    group_by_types = [self.col2type(col) for col in group_by_columns]
    aggregate_types = [agg.__annotations__['return']
                       for agg in aggregates.values()]
    result_table = Table(new_columns, group_by_types + aggregate_types)

    for key, rows in grouped_rows.items():
        if having(rows):
            new_row = list(key)
            for aggregate_name, aggregate_fn in aggregates.items():
                new_row.append(aggregate_fn(rows))
            result_table.insert(new_row)

    return result_table
```

GROUP BY

- Bây giờ chúng ta hãy thử nghiệm các câu lệnh SQL GROUP BY tương ứng với phương thức group_by trong Table mà ta vừa tạo:

```
def min_user_id(rows) -> int:
    return min(row["user_id"] for row in rows)

def length(rows) -> int:
    return len(rows)

stats_by_length = (
    users
    .select(additional_columns={"name_length" : name_length})
    .group_by(group_by_columns=["name_length"],
              aggregates={"min_user_id" : min_user_id,
                          "num_users" : length})
)
```

- Hãy thử các câu lệnh tương ứng và xem kết quả của bạn !!!!

ORDER BY



5.1.6. ORDER BY

- ORDER BY: xếp thứ tự các hàng được chọn
- Ví dụ: hai tên đầu tiên(theo thứ tự bảng chữ cái) trong users.

```
SELECT * FROM users  
ORDER BY name  
LIMIT 2;
```

ORDER BY



- Ta cũng dễ dàng thêm ORDER BY trong NotQuiteABase

```
def order_by(self, order: Callable[[Row], Any]) -> 'Table':  
    new_table = self.select()          # make a copy  
    new_table.rows.sort(key=order)  
    return new_table
```

JOIN



5.1.7. JOIN

- Các bảng trong SQL không thể lưu trữ được kiểu dữ liệu danh sách. Vì vậy giải pháp chính là tạo một bảng thứ 2 để chứa mối quan hệ giữa các bảng
- Ví dụ : Ta có bảng user_interests để lưu các vấn đề quan tâm của user với 1 cột là interest lưu vấn đề quan tâm của user và 1 cột là user_id là mã của user tương ứng

```
CREATE TABLE user_interests (  
    user_id INT NOT NULL,  
    interest VARCHAR(100) NOT NULL  
);
```

JOIN



- Tương tự như vậy trong NotQuiteABase chúng ta cũng sẽ tạo một đối tượng Table mới để lưu trữ danh sách qua mối quan hệ giữa các Table
- Ví dụ : Ta có một đối tượng Table mới để lưu vấn đề quan tâm của user và insert data cho table đó

```
user_interests = Table(['user_id', 'interest'], [int, str])
user_interests.insert([0, "SQL"])
user_interests.insert([0, "NoSQL"])
user_interests.insert([2, "SQL"])
user_interests.insert([2, "MySQL"])
```

JOIN



- Để phân tích dữ liệu khi chúng tồn tại trên các bảng khác nhau chúng ta cần sử dụng JOIN trong SQL
- JOIN hoạt động bằng cách kết hợp các hàng trong bảng bên trái với các hàng tương ứng trong bảng bên phải
- Ví dụ : Để tìm những user quan tâm đến ngôn ngữ truy vấn 'SQL' ta có câu lệnh SQL sau:

```
SELECT users.name  
FROM users  
JOIN user_interests  
ON users.user_id = user_interests.user_id  
WHERE user_interests.interest = 'SQL'
```

JOIN



- Khi sử dụng JOIN kết quả trả về chỉ chứa các hàng phù hợp ở bảng bên phải ứng với các hàng bảng bên trái
- Ngoài việc lấy được các hàng có giá trị phù hợp ta có thể sử dụng LEFT JOIN để lấy cả những hàng có giá trị không phù hợp. Khi đó các trường lấy từ bảng bên phải sẽ có giá trị NULL
- Ví dụ :

```
SELECT users.id, COUNT(user_interests.interest) AS num_interests
FROM users
LEFT JOIN user_interests
ON users.user_id = user_interests.user_id
```

- LEFT JOIN sẽ đảm bảo người dùng không có interest vẫn có các hàng trong kết quả

JOIN

- Để triển khai JOIN trong NotQuiteABase ta phải kết hợp 2 bảng với nhau.
- Chúng ta sẽ tạo phương thức join với 2 tham số là bảng chúng ta muốn join và một biến boolean `left_join` nếu chúng ta muốn join theo LEFT JOIN
- Sau đó ta sẽ lọc các cột chỉ có ở bảng bên phải bằng cách lấy những cột chỉ có ở bảng bên phải trong tất cả các cột của cả 2 bảng

```
def join(self, other_table: 'Table', left_join: bool = False) -> 'Table':  
  
    join_on_columns = [c for c in self.columns          # columns in  
                        if c in other_table.columns]    # both tables  
  
    additional_columns = [c for c in other_table.columns # columns only  
                           if c not in join_on_columns] # in right table
```

JOIN



- Bước tiếp theo chúng ta tạo một Table mới để lưu các bảng ghi sau khi JOIN.
- Với các cột mới là các cột ta đã lọc ở trên và các cột trong bảng hiện tại cũng với kiểu dữ liệu tương ứng của các cột

```
# all columns from left table + additional_columns from right table
new_columns = self.columns + additional_columns
new_types = self.types + [other_table.col2type(col)
                           for col in additional_columns]

join_table = Table(new_columns, new_types)
```

JOIN

- Cuối cùng chúng ta sẽ duyệt qua các hàng của từng bảng và insert các bản ghi tương ứng với các cột có trong danh sách các cột của bảng kết quả:
- Nếu như `left_join = True` thì chúng ta sẽ insert thêm các giá trị `None` cho các hàng không phù hợp

```
for row in self.rows:
    def is_join(other_row):
        return all(other_row[c] == row[c] for c in join_on_columns)

    other_rows = other_table.where(is_join).rows

    # Each other row that matches this one produces a result row.
    for other_row in other_rows:
        join_table.insert([row[c] for c in self.columns] +
                          [other_row[c] for c in additional_columns])

    # If no rows match and it's a left join, output with Nones.
    if left_join and not other_rows:
        join_table.insert([row[c] for c in self.columns] +
                          [None for c in additional_columns])

return join_table
```

JOIN

- Bây giờ chúng ta có thể sử dụng join để truy vấn những vấn đề quan tâm của user như sau :

```
sql_users = (  
    users  
    .join(user_interests)  
    .where(lambda row: row["interest"] == "SQL")  
    .select(keep_columns=["name"]))  
)
```

- Chúng ta cũng có thể sử dụng LEFT JOIN như sau :

```
def count_interests(rows: List[Row]) -> int:  
    """counts how many rows have non-None interests"""  
    return len([row for row in rows if row["interest"] is not None])  
  
user_interest_counts = (  
    users  
    .join(user_interests, left_join=True)  
    .group_by(group_by_columns=["user_id"],  
              aggregates={"num_interests" : count_interests })  
)
```

Truy vấn con

5.2.1. Truy Vấn Con

- Trong SQL Server, truy vấn con là một truy vấn nằm trong một truy vấn khác. Bạn có thể tạo các truy vấn trong lệnh SQL. Các truy vấn con này nằm trong mệnh đề WHERE, FROM hoặc SELECT.
- Ví dụ:

```
SELECT MIN(user_id) AS min_user_id FROM  
(SELECT user_id FROM user_interests WHERE interest = 'SQL') sql_interests;
```

Truy vấn con

- Đây là truy vấn con được tạo trong NotQuiteABase

```
likes_sql_user_ids = (  
    user_interests  
    .where(lambda row: row["interest"] == "SQL")  
    .select(keep_columns=['user_id'])  
)  
  
likes_sql_user_ids.group_by(group_by_columns=[],  
                             aggregates={"min_user_id" : min_user_id })
```

NoSQL



5.2.2. NoSQL

- NoSQL có nghĩa là Not only SQL
- Không sử dụng mô hình dữ liệu quan hệ
- Mô hình lưu trữ {"key": "value"}
- Hệ thống lưu trữ phân tán
- NoSQL là: hệ cơ sở dữ liệu không ràng buộc, phân tán, mã nguồn mở, khả năng mở rộng theo chiều ngang, có thể chứa hàng petabytes, độ chịu tải và chịu lỗi cao, yêu cầu về tài nguyên phần cứng thấp

So sánh NoSQL với SQL

Tính năng	SQL	NoSQL
Hiệu xuất	Kém hơn NoSQL vì khi truy vấn nó phải tính toán, kiểm tra và xử lý các mối quan hệ trong bảng	Tốt hơn SQL vì nó bỏ qua các ràng buộc
Mở rộng theo chiều ngang	Có thể thực hiện được nhưng quá trình mở rộng sẽ rất phức tạp nếu đã tồn tại dữ liệu trong database	Mở rộng dễ dàng
Tốc độ read/write	Kém hơn NoSQL vì phải đảm bảo tính ràng buộc dữ liệu giữa các bảng. Nếu sử dụng nhiều server thì phải bảo toàn tính nhất quán về dữ liệu ở các server với nhau	Tốc độ nhanh hơn SQL vì bỏ qua cơ chế ràng buộc của các bảng. Vì dữ liệu được lưu trong RAM, sau đó mới đẩy xuống HDD và nó tính nhất quán cuối
Phần cứng	Đòi hỏi phần cứng cao	Không đòi hỏi quá cao phần cứng
Thay đổi số node trong hệ thống	Vì tính nhất quán về dữ liệu nên khi thêm hay xóa 1 node cần phải shutdown hệ thống trong 1 khoảng thời gian	Vì tính nhất quán cuối nên sẽ không cần shutdown hệ thống
Truy vấn và báo cáo	Dễ dàng sử dụng ngôn ngữ SQL query để truy vấn trực tiếp dữ liệu từ database hoặc dùng công cụ hỗ trợ để lấy báo cáo	Việc lấy báo cáo dữ liệu trực tiếp từ NoSQL chưa được hỗ trợ tốt, thực hiện chủ yếu qua giao diện ứng dụng
Mở rộng dữ liệu	Khi muốn bổ xung thêm cột cho một bảng, chúng ta phải khai báo trước	Không cần khai báo trước
Ứng dụng	Sử dụng để xây dựng những hệ thống có quan hệ chặt chẽ và cần tính đồng nhất về dữ liệu như: tài chính, ngân hàng, chứng khoán,...	Sử dụng để xây dựng những hệ thống thông tin lớn, không quá quan trọng về vấn đề đồng nhất dữ liệu trong 1 thời gian nhất định. VD: báo trí, mạng xã hội,...